

Laboratory work 11

Adding CRUD Functionalities

This chapter describes how we can implement **Create, Read, Update, and Delete (CRUD)** functionalities in our frontend. We are going to use the components that we learned about in *Chapter 11, Useful Third-Party Components for React*. We will fetch data from our backend and present the data in a table. Then, we will implement the delete, edit, and create functionalities. In the final part of this chapter, we will add features so that we can export our data to a CSV file.

In this chapter, we will cover the following topics:

- Creating the list page
- Adding the delete functionality
- Adding the add functionality
- Adding the edit functionality
- Exporting the data to CSV

Technical requirements

The Spring Boot cardatabase application that we created in *Chapter 12, Setting Up the Frontend for Our Spring Boot RESTful Web Service*, (the unsecured backend) is required, as is the React app that we created in the same chapter, carfront.

The following GitHub link will also be required:

<https://github.com/PacktPublishing/FullStack-Development-with-Spring-Boot-3-andReact-Fourth-Edition/tree/main/Chapter13>.

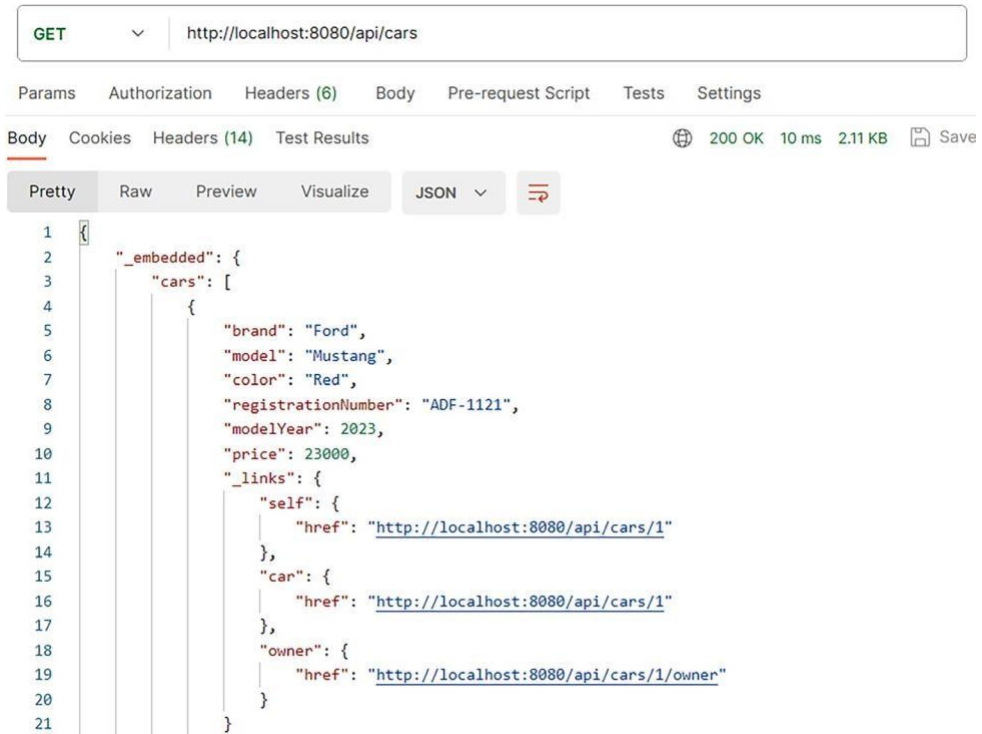
Creating the list page

In this first section, we will create the list page to show cars with paging, filtering, and sorting features:

1. Run your unsecured Spring Boot backend. The cars can be fetched by sending the GET request to the `http://localhost:8080/api/cars` URL, as shown in *Chapter 4*,

Creating a RESTful Web Service with Spring Boot. Now, let's inspect the JSON data from the response.

The array of cars can be found in the `_embedded.cars` node of the JSON response data:



```
GET http://localhost:8080/api/cars

Body  Cookies  Headers (14)  Test Results  200 OK  10 ms  2.11 KB  Save

Pretty  Raw  Preview  Visualize  JSON

1 {
2   "_embedded": {
3     "cars": [
4       {
5         "brand": "Ford",
6         "model": "Mustang",
7         "color": "Red",
8         "registrationNumber": "ADF-1121",
9         "modelYear": 2023,
10        "price": 23000,
11        "_links": {
12          "self": {
13            "href": "http://localhost:8080/api/cars/1"
14          },
15          "car": {
16            "href": "http://localhost:8080/api/cars/1"
17          },
18          "owner": {
19            "href": "http://localhost:8080/api/cars/1/owner"
20          }
21        }
22      }
23    ]
24  }
```

Figure 13.1: Fetching cars

2. Open the car-front React app with Visual Studio Code (the React app we created in the previous chapter).
3. We are using React Query for networking, so we have to initialize the query provider first.



You learned the basics of React Query in *Chapter 10, Consuming the REST API with React*.

The `QueryClientProvider` component is used to connect and provide `QueryClient` to your application. Open your `App.tsx` file and add the highlighted imports and components to your `App` component:

```
import AppBar from '@mui/material/AppBar'; import Toolbar from
 '@mui/material/Toolbar'; import Typography from
 '@mui/material/Typography'; import Container from
 '@mui/material/Container'; import CssBaseline from
 '@mui/material/CssBaseline'; import { QueryClient,
 QueryClientProvider } from '@tanstack/react query';
const queryClient = new QueryClient();
function App() { return
(
  <Container maxWidth="xl">
    <CssBaseline />
    <AppBar position="static">
      <Toolbar>
        <Typography variant="h6">
          Car Shop
        </Typography>
      </Toolbar>
    </AppBar>
    <QueryClientProvider
      client={queryClient}>
    </QueryClientProvider>
  </Container>
)
}

export default App;
```

Now, let's fetch some cars.

Fetching data from the backend

Once we know how to fetch cars from the backend, we will be ready to implement the list page to show the cars. The following steps describe this in practice:

1. When your app has multiple components, it is recommended that you create a folder for them. Create a new folder called `components` in the `src` folder. With Visual Studio Code, you can create a folder by right-clicking the folder in the sidebar file explorer and selecting **New Folder** from the menu:

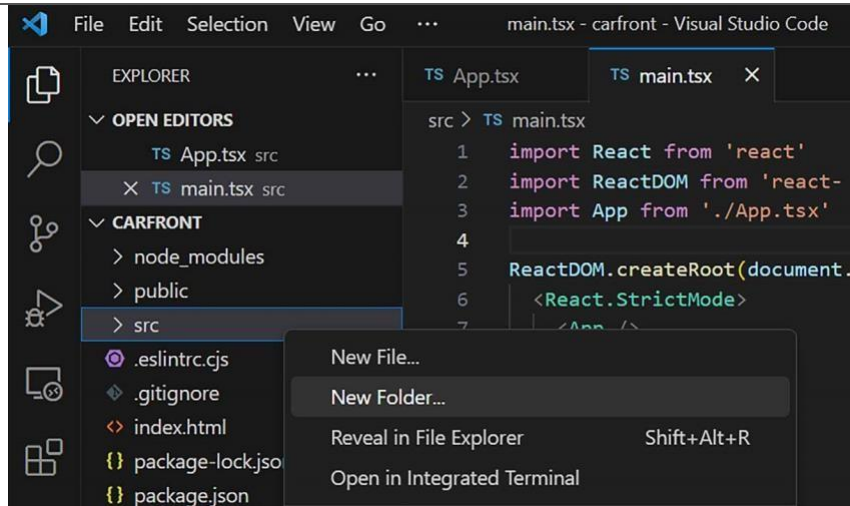


Figure 13.2: New folder

2. Create a new file called `Carlist.tsx` in the components folder. Your project structure should look like the following:

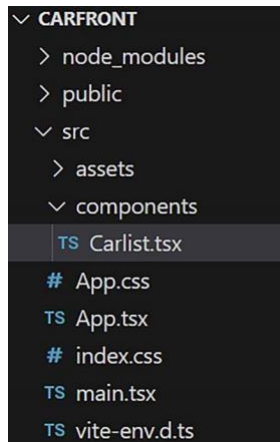


Figure 13.3: Project structure

3. Open the `Carlist.tsx` file in the editor view and write the base code of the component, as follows:

```
function Carlist()
{
  return(
    <></>
  );
}

export default Carlist;
```

-
- Now, when we are using TypeScript, we have to define the type for our car data. Let's create a new file where we define our types. Create a file called `types.ts` in the `src` folder of your project. From the response, you can see that the car object looks like the following, and it contains all the car properties and also links:

```
{
  "brand": "Ford",
  "model": "Mustang",
  "color": "Red",
  "registrationNumber": "ADF-1121",
  "modelYear": 2023,
  "price": 59000,
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/cars/1"
    },
    "car": {
      "href": "http://localhost:8080/api/cars/1"
    },
    "owner": {
      "href": "http://localhost:8080/api/cars/1/owner"
    }
  }
}
```

- Create the following `CarResponse` type in the `types.ts` file and export it so that we can use it in files where it is needed:

```

export type CarResponse = {
  brand: string;  model: string;
  color: string;
  registrationNumber: string;
  modelYear: number;  price:
  number;  _links: {    self:
  {      href: string;
  },    car: {
  href: string;    },
  owner:
  {      href:
  string;
  }
  };
}

```

6. Next, we will create a function that fetches cars from our backend by sending a GET request to the `http://localhost:8080/api/cars` endpoint. The function returns a **promise** that contains an array of `CarResponse` objects that we defined in our `types.ts` file. We can use the `Promise<Type>` generic, where `Type` indicates the resolved value type of the promise.

Open the `Carlist.tsx` file and add the following imports and function:

```

const getCars = async ():
import { CarResponse } from '../types';
import axios from 'axios';
function Carlist() {
  Promise<CarResponse[]> => {    const response = await
  axios.get("http://localhost:8080/api/
  cars");    return response.data._embedded.cars;
  }

  return(
<></>
  );

```

```
}
```

```
export default Carlist;
```

7. Next, we will use the `useQuery` hook to fetch cars:

```
import { useQuery } from '@tanstack/react-  
query'; import { CarResponse } from '../types';  
import axios from 'axios';  
  
function Carlist() {  const getCars = async ():  
  Promise<CarResponse[]> => {    const response = await  
    axios.get("http://localhost:8080/api/  
cars");    return response.data._embedded.cars;  
  }  
  
  const { data, error, isSuccess } =  
  useQuery({    queryKey: ["cars"],  
  queryFn: getCars  
  });  
  
  return (  
    <></>  
  );  
}
```

```
export default Carlist;
```



The `useQuery` hook uses TypeScript generics because it doesn't fetch data and doesn't know the type of your data. However, React Query can infer the type of the data, so we don't have to do it manually here using generics. If you explicitly set generics, the code looks like this:

```
useQuery<CarResponse[], Error>
```

8. We will use **conditional rendering** to check if the fetch is successful and if there are any errors. If `isSuccess` is false, it means the data fetching is still in progress and, in this case, a loading message is returned. We also check if `error` is true, which indicates there's an error, and an error message is returned. When data is available, we use the `map` function to transform car objects into table rows in the return statement and add the table element:

```
// Carlist.tsx if (!isSuccess)
{
  return
  <span>Loading...</span>
} else if (error) {
  return <span>Error when
fetching cars...</span>
} else {
  return
  (
    <table>
      <tbody>
        {
          data.map((car: CarResponse) =>
            <tr key={car._links.self.href}>
              <td>{car.brand}</td>
              <td>{car.model}</td>
              <td>{car.color}</td>
              <td>{car.registrationNumber}</td>
              <td>{car.modelYear}</td>
              <td>{car.price}</td>
            </tr>
          )
        }
      </tbody>
    </table>
  );
}
```

9. Finally, we have to import and render the Carlist component in our App.tsx file. In the App.tsx file, add the import statement, and then render the Carlist component inside the QueryClientProvider component, as highlighted. QueryClientProvider is a component that provides the React Query context to your components, and it should wrap the components where you are making REST API requests from:


```

import AppBar from '@mui/material/AppBar'; import Toolbar from
 '@mui/material/Toolbar'; import Typography from
 '@mui/material/Typography'; import Container from
 '@mui/material/Container'; import CssBaseline from
 '@mui/material/CssBaseline'; import { QueryClient,
 QueryClientProvider } from '@tanstack/react query'; import
 Carlist from './components/Carlist';
const queryClient = new QueryClient();

function App() { return
(
  <Container maxWidth="xl">
    <CssBaseline />
    <AppBar position="static">
<Toolbar>
      <Typography variant="h6">
        Car shop
      </Typography>
    </Toolbar>
  </AppBar>
  <QueryClientProvider client={queryClient}>
    <Carlist />
  </QueryClientProvider>
</Container>
)
}

export default App;

```

10. Now, if you start the React app using the `npm run dev` command, you should see the following list page. Note that your backend should also be running:

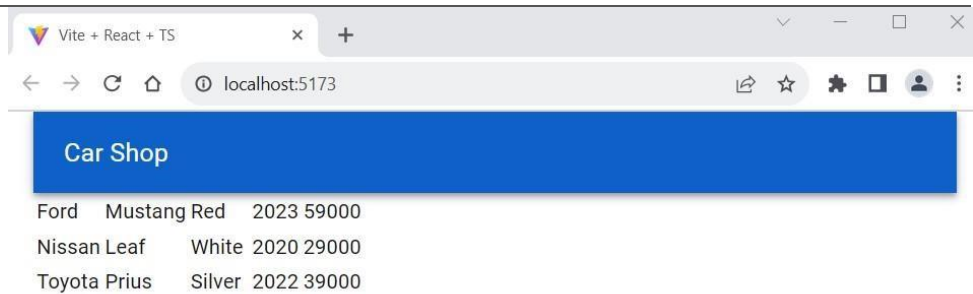


Figure 13.4: Car frontend

Using environment variables

Let's do some code refactoring before we move on. The server URL can be repeated multiple times in the source code when we create more CRUD functionalities, and it will change when the backend is deployed to a server other than the local host; therefore, it is better to define it as an **environment variable**. Then, when the URL value changes, we only have to modify it in one place.

When using Vite, environment variable names should start with the text `VITE_`. Only variables prefixed with `VITE_` are exposed to your source code:

1. Create a new `.env` file in the root folder of our app. Open the file in the editor and add the following line to the file:

```
VITE_API_URL=http://localhost:8080
```

2. We will also separate all API call functions into their own module. Create a new folder named `api` in the `src` folder of your project. Create a new file called `carapi.ts` in the `api` folder, and now your project structure should look like the following:

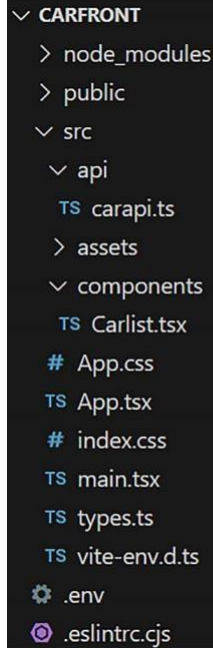


Figure 13.5: Project structure

3. Copy the `getCars` function from the `Carlist.tsx` file to the `carapi.ts` file. Add `export` at the beginning of the function so that we can use it in other components. In Vite, the environment variables are exposed to your app source code via `import.meta.env` as strings. Then, we can import the server URL environment variable to our `getCars` function and use it there. We also need to import `axios` and the `CarResponse` type into the `carapi.ts` file:

```
// carapi.ts import { CarResponse } from
'../types'; import axios from
'axios';

export const getCars = async (): Promise<CarResponse[]> => { const
response = await axios.get(`${import.meta.env.VITE_API_URL}/
api/cars`); return response.data._embedded.cars;
}
```

4. Now, we can remove the `getCars` function and unused `axios` import from the `Carlist.tsx` file and import it from the `carapi.ts` file. The source code should appear as follows:

```

// Carlist.tsx // Remove getCars function and import it from
carapi.ts import { useQuery } from
'@tanstack/react-query'; import { getCars } from
'../api/carapi';
function Carlist() { const { data, error, isSuccess
} = useQuery({ queryKey:
["cars"], queryFn: getCars
});

    if (!isSuccess) { return
<span>Loading...</span>
    } else if (error) { return <span>Error when
fetching cars...</span>
    } else { return
(
<table>
    <tbody> { data.map((car:
CarResponse) =>
<tr key={car._links.self.href}>
        <td>{car.brand}</td>
        <td>{car.model}</td>
        <td>{car.color}</td>
        <td>{car.registrationNumber}</td>
        <td>{car.modelYear}</td>
        <td>{car.price}</td>
    </tr>)
    }
    </tbody>
</table>

);
}
}

```

```
export default Carlist;
```

After these refactoring steps, you should see the car list page like previously.

Adding paging, filtering, and sorting

We have already used the `ag-grid` component to implement a data grid in *Chapter 11, Useful Third-Party Components for React*, and it could be used here as well. Instead, we will use the new MUI `DataGrid` component to get paging, filtering, and sorting features out of the box:

1. Stop the development server by pressing `Ctrl + C` in the terminal.
2. We will install the MUI data grid community version. The following is the installation command at the time of writing, but you should check the latest installation command and usage from the MUI documentation:

```
npm install @mui/x-data-grid
```

3. After installation, restart the app.
4. Import the `DataGrid` component into your `Carlist.tsx` file. We will also import `GridColDef`, which is a type for the column definitions in the MUI data grid:

```
import { DataGrid, GridColDef } from '@mui/x-data-grid';
```

5. The grid columns are defined in the `columns` variable, which has the type `GridColDef[]`. The column `field` property defines where data in the columns is coming from; we are using our car object properties. The `headerName` prop can be used to set the title of the columns. We will also set the width of the columns. Add the following column definition code inside the `Carlist` component:

```
const columns: GridColDef[] = [
  {field: 'brand', headerName: 'Brand', width: 200},
  {field: 'model', headerName: 'Model', width: 200},
  {field: 'color', headerName: 'Color', width: 200},
  {field: 'registrationNumber', headerName: 'Reg.nr.', width: 150},
  {field: 'modelYear', headerName: 'Model Year', width: 150},
  {field: 'price', headerName: 'Price', width: 150}, ];
```

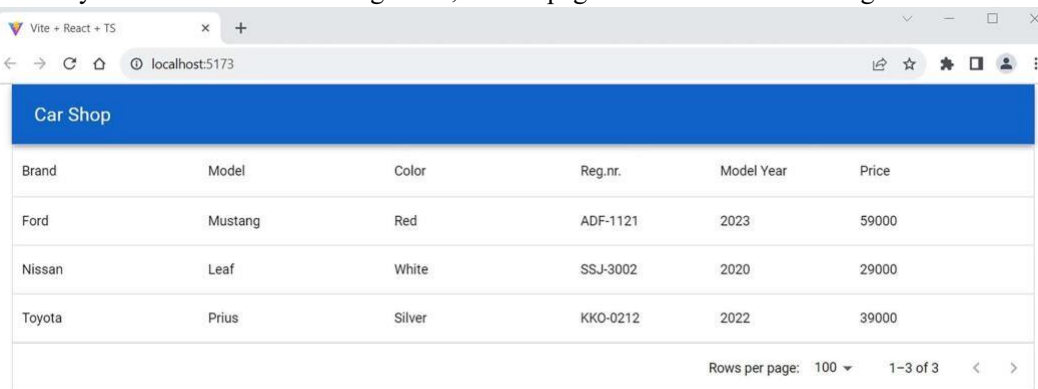
6. Then, remove the table and all its child elements from the component's return statement and add the `DataGrid` component. Also remove the unused `CarResponse` import that we used in the `map` function. The data source of the data grid is the `data`, which contains fetched cars and is defined using the `rows` prop. The `DataGrid` component requires all rows to have a unique ID property that is defined using the `getRowId` prop. We can use the `link` field of the car object because that contains the unique car ID (`_links.self.href`). Refer to the source code of the following return statement:

```

if (!isSuccess) { return
<span>Loading...</span>
} else if (error) { return <span>Error when fetching
cars...</span>
} else { return (
  <DataGrid
    rows={data} columns={columns}
    getRowId={row
row._links.self.href}
    =>
  />
);
}

```

With the MUI DataGrid component, we implemented all the necessary features for our table with only a small amount of coding. Now, the list page looks like the following:



Brand	Model	Color	Reg.nr.	Model Year	Price
Ford	Mustang	Red	ADF-1121	2023	59000
Nissan	Leaf	White	SSJ-3002	2020	29000
Toyota	Prius	Silver	KKO-0212	2022	39000

Rows per page: 100 1-3 of 3

Figure 13.6: Car frontend

Data grid columns can be filtered using the column menu and clicking the **Filter** menu item. You can also set the visibility of the columns from the column menu:

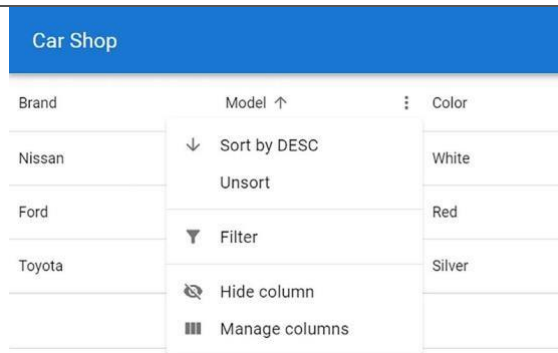


Figure 13.7: Column menu

Next, we will implement the delete functionality.

Adding the delete functionality

Items can be deleted from the database by sending the DELETE method request to the `http://localhost:8080/api/cars/{carId}` endpoint. If we look at the JSON response data, we can see that each car contains a link to itself, which can be accessed from the `_links.self.href` node, as shown in the following screenshot:

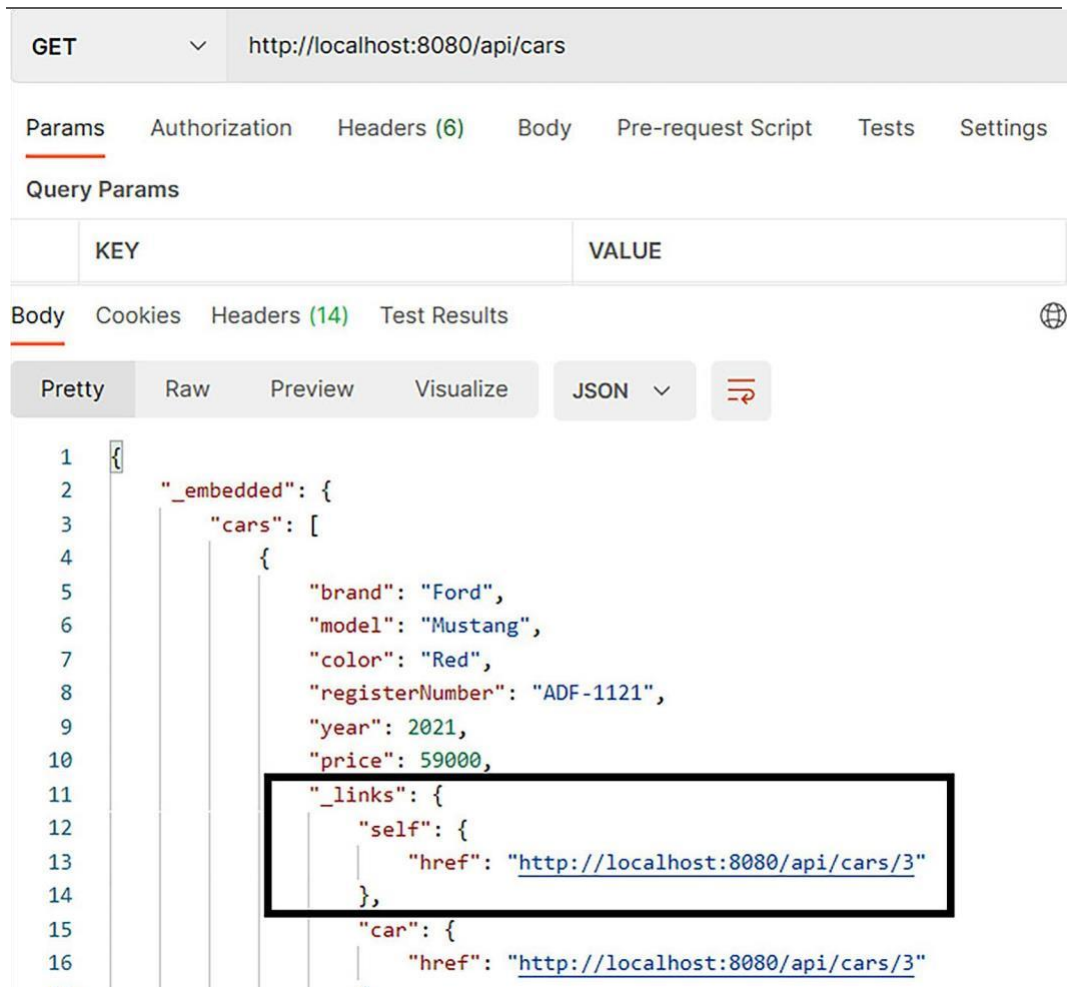


Figure 13.8: Car link

We already used the `link` field in the previous section to set a unique ID for every row in the grid. That row ID can be used in deletion, as we will see later.

The following steps demonstrate how to implement the delete functionality:

1. First, we will create a button for each row in the MUI DataGrid. When we need more complex cell content, we can use the `renderCell` column property to define how a cell's contents are rendered.

Let's add a new column to the table using `renderCell` to render the button element. The `params` argument that is passed to the function is a row object that contains all values from a row. The type of `params` is `GridCellParams`, which is provided by MUI. In our case, it contains a link to a car in each row, and that is needed in the deletion. The link is in the row's `_links.self.href` property, and we will pass this value to a delete

function. Let's first show an alert with the ID when a button is pressed to test that the button is working properly. Refer to the following source code:

```
// Import GridCellParams import { DataGrid, GridColDef,
GridCellParams } from '@mui/x-data grid';

// Add delete button column to columns const
columns: GridColDef[] = [
  {field: 'brand', headerName: 'Brand', width: 200},
  {field: 'model', headerName: 'Model', width: 200},
  {field: 'color', headerName: 'Color', width: 200},
  {field: 'registrationNumber', headerName: 'Reg.nr.', width: 150},
  {field: 'modelYear', headerName: 'Model Year', width: 150},
  {field: 'price', headerName: 'Price', width: 150},
  {
    field: 'delete',
    headerName:
    '',
    width: 90,
    sortable: false,
    filterable: false,
    disableColumnMenu:
    true,
    renderCell: (params:
    GridCellParams) => (
      <button onClick={() =>
        alert(params.row._links.car.href)}
      >
        Delete
      </button>
    ),
  },
];
```

We don't want to enable sorting and filtering for the button column, so the `filterable` and `sortable` props are set to `false`. We also disable the column menu in this column by setting the `disableColumnMenu` prop to `true`. The button invokes the `onDeleteClick` function when pressed and passes a link (`row.id`) to the function as an argument, and the link value is shown in an alert.

2. Now, you should see a **delete** button in each row. If you press any of the buttons, you can see an alert that shows the link for the car. To delete a car, we should send a `DELETE` request to its link:

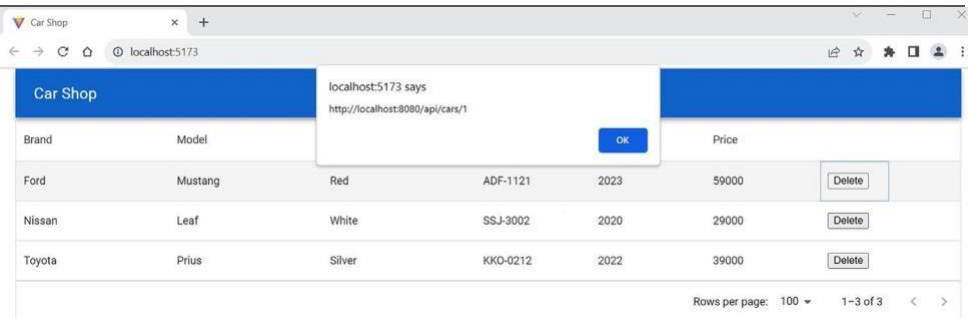


Figure 13.9: Delete button

- Next, we will implement the `deleteCar` function, which sends the `DELETE` request to a car link using the `Axios` `delete` method. A `DELETE` request to the backend returns a deleted car object. We will implement the `deleteCar` function in the `carapi.ts` file and export it. Open the `carapi.ts` file and add the following function there:

```
// carapi.ts export const deleteCar = async (link: string):
Promise<CarResponse> => {
  const response = await
  axios.delete(link);  return response.data
}
```

- We use the `React Query` `useMutation` hook to handle deletion. We saw an example in *Chapter 10*. First, we have to add the `useMutation` import to the `Carlist.tsx` file. We will also import the `deleteCar` function from the `carapi.ts` file:

```
// Carlist.tsx import { useQuery, useMutation } from
'@tanstack/react-query'; import { getCars, deleteCar } from
'../api/carapi';
```

- Add the `useMutation` hook, which calls our `deleteCar` function:

```
// Carlist.tsx const { mutate } = useMutation(deleteCar,
{  onSuccess: ()
=> {
    // Car deleted
  },  onError: (err)
=> {
  console.error(err);
  },
});
```

- Then, call `mutate` in our delete button and pass the car link as an argument:

```
// Carlist.tsx columns {      field: 'delete',      headerName:
'',      width: 90,      sortable:
false,      filterable: false,
disableColumnMenu: true,      renderCell: (params:
GridCellParams) => (
    <button      onClick={() =>
mutate(params.row._links.car.href)      }
    >
    Delete
    </button>
),
},
});
```

7. Now, if you start the app and press the **delete** button, the car is deleted from the database, but it still exists in the frontend. You can manually refresh the browser, after which the car disappears from the table.
8. We can also refresh the frontend automatically when a car is deleted. In React Query, the fetched data is saved to a cache that the query client handles. The QueryClient has a **query invalidation** feature that we can use to fetch data again. First, we have to import and call the useQueryClient hook function, which returns the current query client:

```
// Carlist.tsx import { useQuery, useMutation, }useQueryClient
from '@tanstack/      react-query'; import { deleteCar } from
'../api/carapi'; import { DataGrid, GridColDef, GridCellParams }
from '@mui/x-data      grid';

function Carlist() {      const
queryClient = useQueryClient();
      // continue...
```

9. The queryClient has an invalidateQueries method that we can call to re-fetch our data after successful deletion. You can pass the key of the query that you want to refetch. Our query key for fetching cars is cars, which we defined in our useQuery hook:

```
// Carlist.tsx const { mutate } = useMutation(deleteCar, {
  onSuccess: () => {      queryClient.invalidateQueries({
    queryKey: ['cars'] });
  },      onError: (err)
=> {      console.error(err);
  },
});
```

Now, every time a car is deleted, all the cars are fetched again. The car disappears from the list when the **Delete** button is pressed. After a deletion, you can restart the backend to re-populate the database.

You can also see that when you click any row in the grid, the row is selected. You can disable that by setting the `disableRowSelectionOnClick` prop in the grid to `true`:

```
<DataGrid rows={cars}
  columns={columns}

  disableRowSelectionOnClick={true}
  getRowId={row => row._links.self}
/>
```

Displaying a toast message

It would be nice to show the user some feedback in the case of a successful deletion, or if there are any errors. Let's implement a **toast message** to show the status of the deletion. For this, we are going to use the MUI Snackbar component:

1. First, we have to import the Snackbar component by adding the following import statement to our Carlist.tsx file:

```
import Snackbar from '@mui/material/Snackbar';
```

2. The Snackbar component's `open` prop value is a boolean, and if it is true, the component is shown; otherwise, it is hidden. Let's import the `useState` hook and define a state called `open` to handle the visibility of our Snackbar component. The initial value is `false` because the message is shown only after the deletion:

```
//CarList.tsx
import { useState } from 'react';
import {
  useQuery,
  useMutation, useQueryClient } from '@tanstack/ react-query';
import { deleteCar } from '../api/carapi'; import { DataGrid,
GridColDef, GridCellParams } from '@mui/x-data grid'; import
Snackbar from '@mui/material/Snackbar';

function Carlist() {
  const [open,
setOpen] = useState(false);
  const queryClient = useQueryClient();
  // continue...
```

3. Next, we add the Snackbar component in the return statement after the MUI DataGrid component. The autoHideDuration prop defines the time in milliseconds after which the onClose function is called automatically and the message disappears. The message prop defines the message to display. We also have to wrap the DataGrid and Snackbar components inside the fragment (<></>):

```
// CarList.tsx if (!isSuccess) { return
<span>Loading...</span>
} else if (error) { return <span>Error when
fetching cars...</span>
} else { return
(
  <>
    <DataGrid rows={data}
columns={columns}
disableRowSelectionOnClick={true}
getRowId={row => row._links.self.href} />
    <Snackbar open={open}
autoHideDuration={2000}
onClose={() =>
setOpen(false)}
message="Car deleted"
/>
  </>
);
```

4. Finally, we set the open state to true after the successful deletion in our useMutation hook:

```
// CarList.tsx const { mutate } =
useMutation(deleteCar, { onSuccess: () =>
{ setOpen(true);
queryClient.invalidateQueries(["cars"]);
}, onError: (err)
=> {
console.error(err);
},
});
```

Now, you will see the toast message when the car is deleted, as shown in the following screenshot:

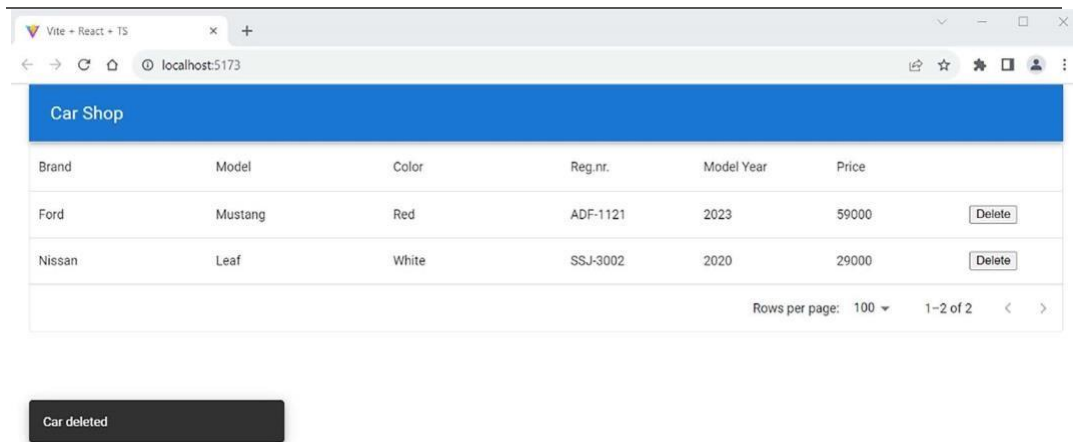


Figure 13.10: Toast message

Adding a confirmation dialog window

To avoid accidental deletion of a car, it would be useful to have a confirmation dialog after the **Delete** button has been pressed. We will implement this using the `window` object's `confirm` method. It opens a dialog with an optional message and returns `true` if you press the **OK** button. Add `confirm` to the delete button's `onClick` event handler:

```

false, filterable: false,
disableColumnMenu: true, renderCell:
(params: GridCellParams) => (
  <button onClick={() => { if (window.confirm(`Are you
sure you want to delete ${params.row.
brand} ${params.row.model}?`)) {
mutate(params.row._links.car.href);
}
}}
>
  Delete
</button>
// Carlist.tsx columns { field: 'delete', headerName:
'', width: 90, sortable:
),
}

```

In the confirmation message, we have used ES6 string interpolation to display the car's brand and model. (Note! Remember to use backticks.)

If you press the **Delete** button now, the confirmation dialog will open and the car will only be deleted if you press the **OK** button:

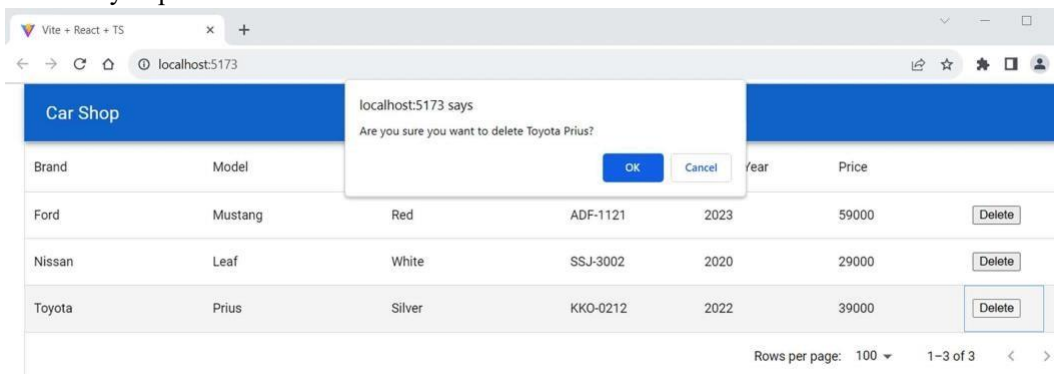


Figure 13.11: Confirmation dialog

Next, we will begin the implementation of the functionality to add a new car.

Adding the add functionality

The next step is to add an add functionality to the frontend. We will implement this using the MUI modal dialog.



We went through the MUI modal form in *Chapter 11 Useful Third-Party Components for React*.

We will add the **New Car** button to the user interface, which opens the modal form when it is pressed. The modal form contains all the fields that are required to add a new car, as well as the buttons for saving and canceling.

The following steps show you how to create the add functionality using the modal dialog component:

1. Create a new file called `AddCar.tsx` in the `components` folder and write some functional component base code to the file, as shown here. Add the imports for the MUI Dialog component:

```
import Dialog from '@mui/material/Dialog'; import
DialogActions from '@mui/material/DialogActions';
import DialogContent from
'@mui/material/DialogContent'; import DialogTitle from
'@mui/material/DialogTitle';

function AddCar() {  return(
<></>
);
}

export default AddCar;
```

2. We have already defined the type for our Car response data (a car object with links). Let's also create a type for the car object that doesn't contain links, because the user doesn't enter links in the form. We need this for the state where we will save a new car. Add the following Car type to your `types.ts` file:

```
export type Car = {  brand: string;  model: string;
color: string;  registrationNumber: string;  modelYear:
number;  price: number;
}
```

3. Declare a state of type Car that contains all car fields using the useState hook. For the dialog, we also need a boolean state to define the visibility of the dialog form:

```
import { useState } from 'react'; import Dialog from
 '@mui/material/Dialog'; import DialogActions from
 '@mui/material/DialogActions'; import DialogContent
 from '@mui/material/DialogContent'; import DialogTitle
 from '@mui/material/DialogTitle'; import { Car } from
 '../types';
function AddCar() { const [open,
  setOpen] = useState(false); const [car,
  setCar] = useState<Car>({ brand: '',
  model: '', color: '',
  registrationNumber: '', modelYear: 0,
  price: 0
}); return(
<></>
); } export default
AddCar;
```

4. Next, we add two functions to close and open the dialog form. The handleClose and handleOpen functions set the value of the open state, which affects the visibility of the modal form:

```
// AddCar.tsx // Open the modal
form const handleClickOpen = ()
=> { setOpen(true);
};
// Close the modal form
const handleClose = () => {
  setOpen(false);
};
```

5. Add the Dialog component inside the AddCar component's return statement. The form contains the MUI Dialog component with buttons and the input fields that are required to collect the car data. The button that opens the modal window, which will be shown on the car list page, must be outside of the Dialog component. All input fields should have a name attribute with a value that is the same as the name of the state the value will be saved to. Input fields also have the onChange prop, which saves the value to the car state by invoking the handleChange function. The handleChange function dynamically updates the car state by creating a new object with the existing state properties and updating a property based on the input element's name and the new value entered by the user:

```
// AddCar.tsx    const handleChange = (event :
React.ChangeEvent<HTMLInputElement>) =>
{   setCar({...car, [event.target.name]:
    event.target.value});
}

return(
  <>
    <button onClick={handleClickOpen}>New Car</button>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>New car</DialogTitle>
      <DialogContent>
        <input placeholder="Brand" name="brand"
value={car.brand} onChange={handleChange}/><br/>
        <input placeholder="Model" name="model"
value={car.model} onChange={handleChange}/><br/>
        <input placeholder="Color" name="color"
value={car.color} onChange={handleChange}/><br/>
        <input placeholder="Year" name="modelYear"
value={car.modelYear} onChange={handleChange}/><br/>
        <input placeholder="Reg.nr" name="registrationNumber"
value={car.registrationNumber} onChange={handleChange}/><br/>
        <input placeholder="Price" name="price"
value={car.price} onChange={handleChange}/><br/>
      </DialogContent>
      <DialogActions>
        <button onClick={handleClose}>Cancel</button>
        <button onClick={handleClose}>Save</button>
      </DialogActions>
    </Dialog>
  </>
);
```

-
6. Implement the `addCar` function in the `carapi.ts` file, which will send the POST request to the backend `api/cars` endpoint. We are using the Axios post method to send POST requests. The request will include the new car object inside the body and the `'Content-Type': 'application/json'` header. We also need to import the `Car` type because we are passing a new car object as an argument to the function:

```
// carapi.ts import { CarResponse, Car } from
'../types';

// Add a new car export const addCar = async (car: Car):
Promise<CarResponse> => {  const response = await
axios.post(`${import.meta.env.VITE_API_
URL}/api/cars`, car, {  headers: {
  'Content-Type': 'application/json',
},
});  return response.data;
}
```

7. Next, we use the React Query `useMutation` hook, like we did in the delete functionality. We also invalidate the cars query after the car has been added successfully. The `addCar` function that we use in the `useMutation` hook is imported from the `carapi.ts` file. Add the following imports and the `useMutation` hook to your `AddCar.tsx` file. We also need to get the query client from the context using the `useQueryClient` hook. Remember that context is used to provide access to the query client to components deep in the component tree:

```
// AddCar.tsx // Add the following imports import { useMutation,
useQueryClient } from '@tanstack/react-query'; import { addCar } from
'../api/carapi';

// Add inside the AddCar component function const
queryClient = useQueryClient();

// Add inside the AddCar component function const
{ mutate } = useMutation(addCar, { onSuccess:
() => {
queryClient.invalidateQueries(["cars"]);
}, onError: (err)
=> { console.error(err);
},
});
```

8. Import the AddCar component into the Carlist.tsx file:

```
// Carlist.tsx import AddCar
from './AddCar';
```

9. Add the AddCar component to the Carlist.tsx file's return statement. You also have to import the AddCar component. Now, the return statement of the Carlist.tsx file should appear as follows:

```
// Carlist.tsx // Add the following
import import AddCar from
'./AddCar';

// Render the AddCar component return
(
  <>
    <AddCar /> <DataGrid
rows={data} columns={columns}
disableRowSelectionOnClick={true}
```

```

      getId={row => row._links.self.href}/>
    <Snackbar open={open}
      autoHideDuration={2000} onClose={()
=> setOpen(false)} message="Car
deleted" />
  </>
);

```

10. If you start the car shop app, it should now look like the following:

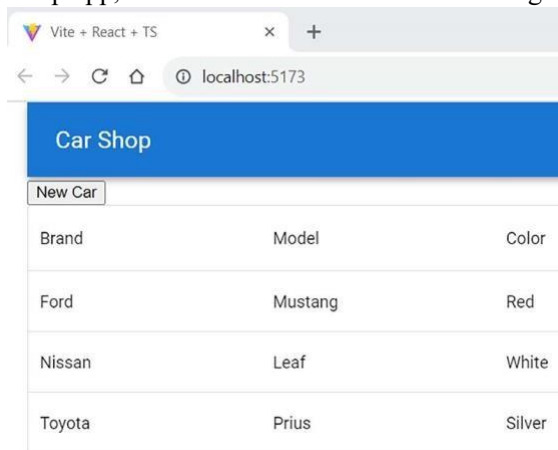


Figure 13.12: Car Shop

If you press the **New Car** button, it should open the modal form.

11. To save a new car, create a function called `handleSave` in the `AddCar.tsx` file. The `handleSave` function calls `mutate`. Then, we set the car state back to its initial state, and the modal form is closed:

```

// AddCar.tsx // Save car and close modal form const handleSave =
() => { mutate(car); setCar({ brand: '', model: '', color:
'', registrationNumber:'', modelYear: 0, price: 0 });
handleClose();
}

```

12. Finally, we have to change the `AddCar` component's `onClick` save button to call the `handleSave` function:

```

// AddCar.tsx
<DialogActions>

```

```

<button onClick={handleClose}>Cancel</button>
<button handleSave onClick={}>Save</button>
</DialogActions>

```

13. Now, you can open the modal form by pressing the **New Car** button. You will see that there is placeholder text in each field when it is empty. You can fill out the form with data and press the **Save** button. At this point, the form doesn't have a nice appearance, but we are going to style it in the next chapter:

Color	Reg.nr.	Model Year
Red		2023
White		2020
Silver		2022

Figure 13.13: Add new car

14. After saving, the list page is refreshed, and the new car can be seen in the list:

Brand	Model	Color	Reg.nr.	Model Year	Price	
Ford	Mustang	Red	ADF-1121	2023	59000	Delete
Nissan	Leaf	White	SSI-3002	2020	29000	Delete
Toyota	Prius	Silver	KKO-0212	2022	39000	Delete
Volkswagen	Beetle	Yellow	ABC-313	2024	45000	Delete

Rows per page: 100 1-4 of 4

Figure 13.14: Car Shop

15. Now, we can do some code refactoring. When we start to implement the edit functionality, we will actually need the same fields in the **Edit** form as in the **New Car** form. Let's create a new component that renders the text fields in our **New Car** form. The idea is that we are splitting the text fields into their own component, which can then be used in both the **New Car** and **Edit** forms. Create a new file called `CarDialogContent.tsx` in the components folder. We have to pass the car object and the `handleChange` function to the component

using props. To do that, we create a new type called `DialogFormProps`. We can define this type in the same file because we don't need it in any other file:

```
// CarDialogContent.tsx import {
  Car } from '../types';

type DialogFormProps = {   car: Car;   handleChange: (event:
  React.ChangeEvent<HTMLInputElement>) =>   void;
}

function CarDialogContent({ car, handleChange }: DialogFormProps) {
  return (
    <></>
  );
}

export default CarDialogContent;
```

16. Then, we can move our `DialogContent` component from the `AddCar` component to the `CarDialogContent` component. Your code should look like the following:

```
// CarDialogContent.tsx import DialogContent from
  '@mui/material/DialogContent' ; import { Car } from
  '../types';

type DialogFormProps = {   car: Car;   handleChange: (event:
  React.ChangeEvent<HTMLInputElement>) =>   void;
}
```



```

function CarDialogContent({ car, handleChange}: DialogFormProps) {
  return (
    <DialogContent>
      <input
        placeholder="Brand"
        name="brand"
        value={car.brand}
        onChange={handleChange}/><br/>
      <input
        placeholder="Model"
        name="model"
        value={car.model}
        onChange={handleChange}/><br/>
      <input
        placeholder="Color"
        name="color"
        value={car.color}
        onChange={handleChange}/><br/>
      <input placeholder="Year"
        name="modelYear"
        value={car.modelYear}
        onChange={handleChange}/><br/>
      <input placeholder="Reg.nr."
        name="registrationNumber"
        value={car.registrationNumber}
        onChange={handleChange}/><br/>
      <input
        placeholder="Price"
        name="price"
        value={car.price}
        onChange={handleChange}/><br/>
    </DialogContent>
  );
}

export default CarDialogContent;

```

17. Now, we can import the CarDialogContent to the AddCar component and render it inside the Dialog component. Pass the car state and the handleChange function to the component using props. Also, remove the unused MUI DialogContent import from the AddCar component:

```
// AddCar.tsx
// Add the following import // and remove unused
imports: DialogContent import CarDialogContent
from './CarDialogContent';

// render CarDialogContent and pass props
return( <div>
  <Button onClick={handleClickOpen}>New Car</Button>
  <Dialog open={open} onClose={handleClose}>
    <DialogTitle>New car</DialogTitle>

    <CarDialogContent car={car} handleChange={handleChange}/>
  <DialogActions>
    <Button onClick={handleClose}>Cancel</Button>
    <Button onClick={handleSave}>Save</Button>
  </DialogActions>
</Dialog>
</div>
);
```

18. Try to add a new car, and it should work like it did before the refactoring.

Next, we will begin to implement the edit functionality.

Adding the edit functionality

We will implement the edit functionality by adding the **Edit** button to each table row. When the row **Edit** button is pressed, it opens a modal form where the user can edit the existing car and save their changes. The idea is that we pass car data from the grid row to the edit form, and the form fields are populated when the form is opened:

1. First, create a file called `EditCar.tsx` in the `components` folder. We have to define a `FormProps` type for our props, and this can be defined inside our component because we don't need this type anywhere else. The type of data that will be passed to the `EditCar` component is the `CarResponse` type. We will also create a state for car data like we did in the add functionality section. The code for the `EditCar.tsx` file looks like the following:

```
// EditCar.tsx import { useState } from
'react'; import { Car, CarResponse } from
'../types';

type FormProps = {   cardata:
CarResponse;
}

function EditCar({ cardata }: FormProps) {
const [car, setCar] = useState<Car>({   brand:
'',   model: '',   color: '',
registrationNumber: '',

    modelYear: 0,   price: 0  });   return(
<></>   );  }

export default EditCar;
```

2. We will create a dialog that will be opened when the **Edit** button is pressed. We need the open state to define if the dialog is visible or hidden. Add the functions that open and close the Dialog component and save updates:

```
// EditCar.tsx import { useState } from 'react'; import
Dialog from '@mui/material/Dialog';
import
DialogActions from
'@mui/material/DialogActions';
import DialogTitle from
'@mui/material/DialogTitle';
import { Car, CarResponse } from '../types';

type FormProps = {   cardata:
CarResponse;
}

function EditCar({ cardata }: FormProps) {
  const [open, setOpen] = useState(false);  const
  [car, setCar] = useState<Car>({    brand: '',
  model: '',    color: '',
  registrationNumber: '',    modelYear: 0,
  price: 0  });
  const handleClickOpen = () => {
```

```

        setOpen(true);
    };
    const handleClose
= () => {
    setOpen(false);
};
    const
handleSave = () => {
    setOpen(false);
}

return(
    <>
        <button onClick={handleClickOpen}>
            Edit
        </button>
        <Dialog open={open}
onClose={handleClose}>
            <DialogTitle>Edit
            car</DialogTitle>
            <DialogActions>
                <button
                    onClick={handleClose}>Cancel</button>
                <button
                    onClick={handleSave}>Save</button>
            </DialogActions>
        </Dialog>
    </>
);
}

export default EditCar;

```

- Next, we will import the CarDialogContent component and render it inside the Dialog component. We also need to add the handleChange function, which saves edited values to the car state. We pass in the car state and the handleChange function using the props, as we did earlier with the add functionality:

```

// EditCar.tsx // Add the following import
import
CarDialogContent from './CarDialogContent';

// Add handleChange function

```

```

const handleChange = (event :
React.ChangeEvent<HTMLInputElement>) => {   setCar({...car,
[event.target.name]: event.target.value}); }

// render CarDialogContent inside the Dialog
return(
  <>
    <button onClick={handleClickOpen}>
      Edit
    </button>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>Edit car</DialogTitle>
      <CarDialogContent car={car} handleChange={handleChange}/>
      <DialogActions>
        <button onClick={handleClose}>Cancel</button>
        <button onClick={handleSave}>Save</button>
      </DialogActions>
    </Dialog>
  </>
);

```

- Now, we will set the values of the car state using the props in the handleClickOpen function:

```

// EditCar.tsx const handleClickOpen = () => {   setCar({
brand: cardata.brand,    model: cardata.model,    color:
cardata.color,    registrationNumber:
cardata.registrationNumber,    modelYear: cardata.modelYear,
price: cardata.price
});

  setOpen(true);
};

```

Our form will be populated with the values from the car object that is passed to the component in props.

- In this step, we will add the edit functionality to our data grid in the Carlist component. Open the Carlist.tsx file and import the EditCar component. Create a new column that renders the EditCar component using the renderCell column property, as we did in the delete functionality section. We pass the row object to the EditCar component,

and that object contains the car object:

```
// CarList.tsx // Add the
following import import EditCar
from './EditCar';

// Add a new column const
columns: GridColDef[] = [
  {field: 'brand', headerName: 'Brand', width: 200},
  {field: 'model', headerName: 'Model', width: 200},
  {field: 'color', headerName: 'Color', width: 200},
  {field: 'registrationNumber', headerName: 'Reg.nr.', width: 150},
  {field: 'modelYear', headerName: 'Model Year', width: 150},
  {field: 'price', headerName: 'Price', width: 150},
  {
    field: 'edit',
    headerName: '',
    width: 90,
    sortable: false,
    filterable: false,
    disableColumnMenu:
    true,
    renderCell: (params:
    GridCellParams) =>
      <EditCar cardata={params.row} />
    },
    {
      field:
      'delete',
      headerName:
      '',
      width: 90,
      sortable: false,
      filterable: false,
      disableColumnMenu: true,
      renderCell: (params: GridCellParams) => (
        <button
          onClick={() => {
            if
            (window.confirm(`Are you sure you want to delete
            ${params.row.brand} ${params.row.model}?`))
            mutate(params.row._links.car.href)
            Delete
          }}>
          </button>
        ),
      },
    ],
  ],
}
```

```
];
```

- Now, you should see the **Edit** button in each table row in your car list. When you press the **Edit** button, it should open the car form and populate fields using the car from the row where you pressed the button:

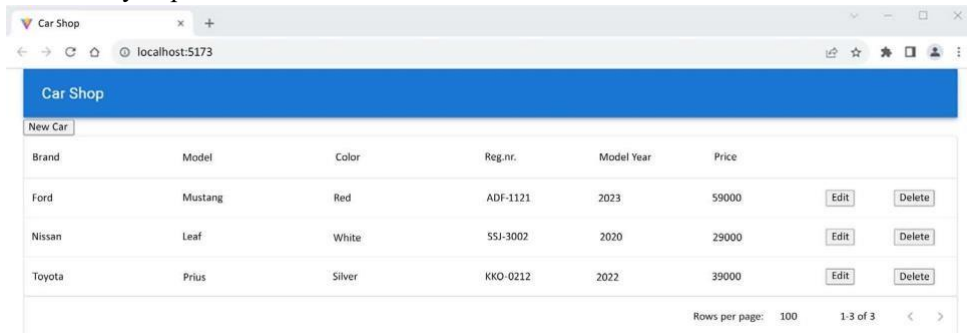


Figure 13.15: Edit button

- Next, we have to implement the update request that sends an updated car to the backend.

To update the car data, we have to send a PUT request to the `http://localhost:8080/api/cars/[carid]` URL. The link will be the same as it is for the delete functionality. The request contains the updated car object inside the body, and the `'Content-Type': 'application/json'` header that we also set for the add functionality. For the update functionality, we need a new type. In React Query, the mutation function can only take one parameter, but in our case, we have to send the car object (Car type) and its link.

We can solve that by passing an object that contains both values. Open the `types.ts` file and create the following type, called `CarEntry`:

```
export type CarEntry = {  car: Car;  url: string;}
```

- Then, open the `carapi.ts` file, create the following function, and export it. The function gets the `CarEntry` type object as an argument and it has `car` and `url` properties, where we get the values that are needed in the request:


```
// carapi.ts // Add CarEntry to import import { CarResponse,
Car, } from CarEntry '../types';

// Add updateCar function export const updateCar = async
(carEntry: CarEntry): Promise<CarResponse> => { const response
= await axios.put(carEntry.url, carEntry.car, { headers: {
  'Content-Type': 'application/json'
}, }); return response.data;
}
```

9. Next, we import the updateCar function into the EditCar component and use the useMutation hook to send a request. We invalidate the cars query to re-fetch the list after a successful edit; therefore, we also have to get the query client:

```
// EditCar.tsx // Add the following imports import { updateCar }
from '../api/carapi'; import { useMutation, useQueryClient } from
'@tanstack/react-query';

// Get query client const queryClient
= useQueryClient();

// Use useMutation hook const { mutate } = useMutation(updateCar,
{ onSuccess: () =>
{ queryClient.invalidateQueries(["cars"]);
}, onError: (err)
=> { console.error(err);
}
});
```

10. Then, we call mutate in the handleSave function. As was already mentioned, mutate only accepts one parameter, and we have to pass the car object and URL; therefore, we create an object that contains both values and pass that one. We also need to import the CarEntry type:

```
// EditCar.tsx // Add CarEntry import import { Car,
CarResponse, CarEntry } from '../types';

// Modify handleSave function const handleSave = () => { const
url} mutate(carEntry); setCar({ brand: '', model: '', color:
url = cardata._links.self.href; const carEntry: CarEntry = {car,
setOpen(false);
'', registrationNumber:'', modelYear: 0, price: 0 });
}
```

11. Finally, if you press the **Edit** button in the table, it opens the modal form and displays the car from that row. The updated values are saved to the database when you press the **Save** button:

Color	Brand	Model Year	Price
Red	Ford	2023	59000
White	Mustang	2020	
Silver	Red	2022	

Edit car

Figure 13.16: Edit car

Similarly, if you press the **New Car** button, it will open an empty form and save the new car to the database when the form is filled and the **Save** button is pressed. We used one component to handle both use cases by using the component props.

12. You can also see what happens in the backend when you edit a car. If you look at the Eclipse console after a successful edit, you can see that there is an update SQL statement that updates the database:

```
Console x JUnit
CardatabaseApplication [Java Application] C:\Program Files\Java\jdk-17.0.2\bin\javaw.exe (28.9.2023 klo 8.24.28) [pid: 21388]
Hibernate: select c1_0.id,c1_0.brand,c1_0.color,c1_0.model,c1_0.model_year,c1_0.owner,c1_0.price,c1_0.registrati
Hibernate: select c1_0.id,c1_0.brand,c1_0.color,c1_0.model,c1_0.model_year,c1_0.owner,c1_0.price,c1_0.registrati
Hibernate: update car set brand=?,color=?,model=?,model_year=?,owner=?,price=?,registration_number=? where id=?
```

Figure 13.17: Update car statement

Now, we have implemented all the CRUD functionalities.

Exporting the data to CSV

One feature that we will also implement is a **comma-separated values (CSV)** export of the data. We don't need any extra libraries for the export because the MUI data grid provides this feature.

We will activate the data grid toolbar, which contains a lot of nice features:

1. Add the following import to the `Carlist.tsx` file. The `GridToolbar` component is a toolbar for the MUI data grid that contains nice functionalities, such as export:

```
import {   DataGrid,
          GridColDef,
          GridCellParams,
          GridToolbar
        } from '@mui/x-data-grid';
```

2. We need to enable our toolbar, which contains the **Export** button and other buttons. To enable the toolbar in the MUI data grid, you have to use the `slots` prop and set the value to `toolbar: GridToolbar`. The `slots` prop can be used to override the data grid's internal components:

```
return(
  <>
    <AddCar />    <DataGrid      rows={cars}
                  columns={columns}
                  disableRowSelectionOnClick={true}
                  getRowId={row => row._links.self.href}
                  slots={{ toolbar: GridToolbar }}
                  />    <Snackbar      open={open}
                      autoHideDuration={2000}
                      onClose={() => setOpen(false)}
                      message="Car deleted"
                      />
  </>
);
```

- Now, you will see the **EXPORT** button in the grid. If you press the button and select **Download as CSV**, the grid data is exported to a CSV file. You can **Print** your grid using the **EXPORT** button, and you will get a printer-friendly version of your page (you can also hide and filter columns and set row density using the toolbar):

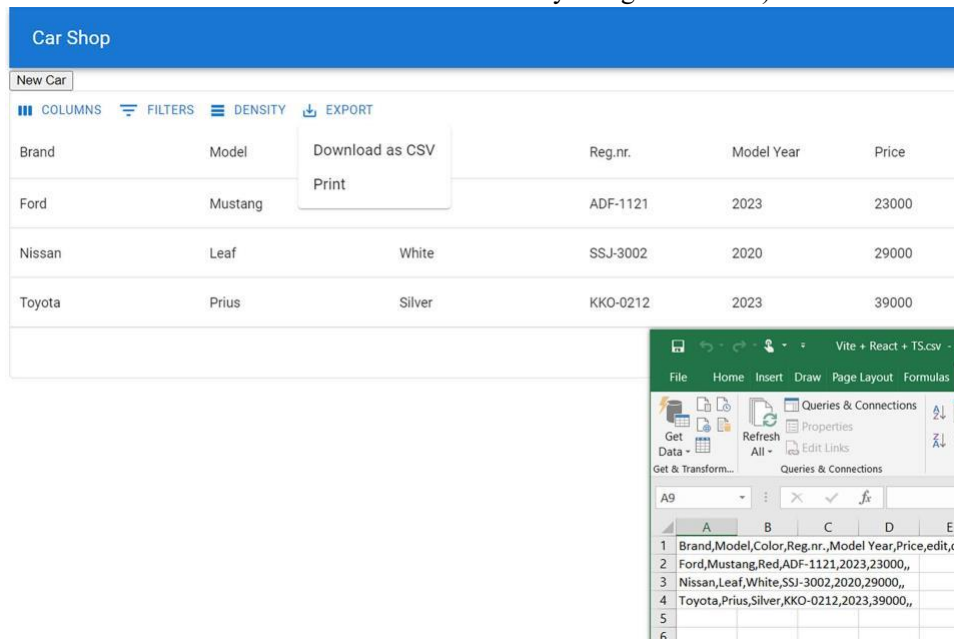


Figure 13.18: Export CSV

- You can change the page title and icon by editing the `index.html` page, as shown in the following code. The icon can be found in your project's public folder, and you can use your own icon instead of Vite's default one:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial
scale=1.0" />    <title>Car Shop</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Now, all the functionalities have been implemented.